

✓ BPL_TEST2_Batch script with OMSimulator

The key library OMSimulator is installed.

After the installation a small application BPL_TEST2_Batch is loaded and run. You can continue with this example if you like.

```
!lsb_release -a # Actual VM Ubuntu version used by Google
```

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 24.04.5 LTS
Release:      24.04
Codename:     noble
```

```
!python --version
```

```
Python 3.13.15
```

```
!pip install OMSimulator
```

```
Collecting OMSimulator
  Downloading omsimulator-3.0.0.post116.tar.gz (7.6 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (2.32.4)
Requirement already satisfied: lxml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (6.1.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (2025.11.12)
Building wheels for collected packages: OMSimulator
  Building wheel for OMSimulator (pyproject.toml) ... done
  Created wheel for OMSimulator: filename=omsimulator-3.0.0.post116-py3-none-any.whl size=1345325 sha256=cfa3e3b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1b1
  Stored in directory: /root/.cache/pip/wheels/eb/b0/ee/5194adbcfe75a2efc0262c5c0377b28ecbc648628be707b5a4
Successfully built OMSimulator
Installing collected packages: OMSimulator
Successfully installed OMSimulator-3.0.0.post116
```

✓ BPL_TEST2_Batch setup

Now specific installation and the run simulations. Start with connecting to Github. Then upload the two files:

- FMU - BPL_TEST2_Batch_linux_om_me.fmu
- Setup-file - BPL_TEST2_Batch_fmpy_explore.py

```
%%bash
git clone https://github.com/janpeter19/BPL_TEST2_Batch
```

```
Cloning into 'BPL_TEST2_Batch'...
```

```
%%cd BPL_TEST2_Batch
```

```
/content/BPL_TEST2_Batch
```

✓ BPL_TEST2_Batch - demo

```
import OMSimulator as oms
```

```
import scipy.io
```

```
import matplotlib.pyplot as plt
```

```
fmu = oms.FMU('BPL_TEST2_Batch_linux_om_me.fmu')
```

```
fmu.modelName
```

```
'BPL.Examples_TEST2.Batch'
```

```
for s in fmu.states:  
    print(s.name)
```

```
bioreactor.V  
bioreactor.m[1]  
bioreactor.m[2]
```

```
fmu.instantiate()
```

```
fmu.setResultFile('result.mat')
```

```
fmu.setValue('bioreactor.V_start', 1.0)  
fmu.setValue('bioreactor.m_start[1]', 1.0)  
fmu.setValue('bioreactor.m_start[2]', 10.0)
```

```
fmu.initialize()
```

```
fmu.simulate()
```

```
mat_data = scipy.io.loadmat('result.mat')
```

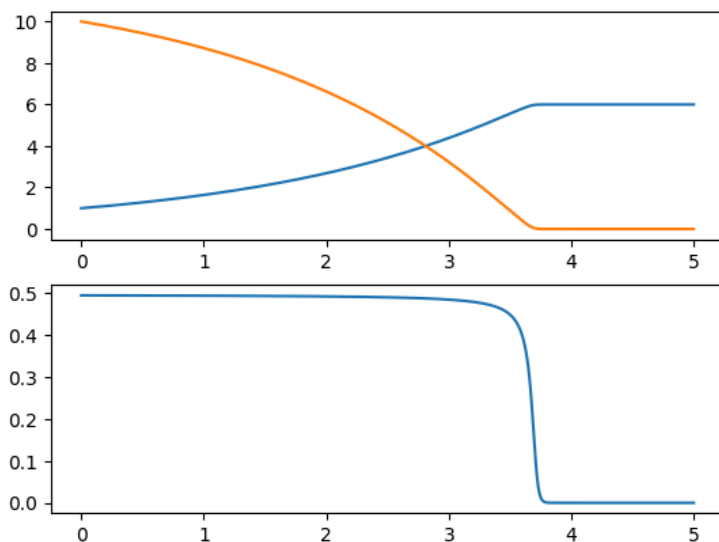
```
mat_data.keys()
```

```
dict_keys(['Aclass', 'name', 'description', 'dataInfo', 'data_1', 'data_2'])
```

```
t = mat_data['data_2'][0];
```

```
plt.figure()  
  
ax1 = plt.subplot(2,1,1)  
ax2 = plt.subplot(2,1,2)  
  
ax1.plot(t, mat_data['data_2'][2])  
ax1.plot(t, mat_data['data_2'][3])  
ax2.plot(t, mat_data['data_2'][12])
```

[<matplotlib.lines.Line2D at 0x7fd21ad6f9d0>]



Simulation looks what I expected!

Start coding or [generate](#) with AI.

